

Microcontroller family 8051

Instruction set

Abbreviations

<i>A</i> :	Accumulator
<i>Rn</i> :	Register R0..R7
<i>Ri</i> :	R0 or R1
<i>dadr</i> :	direct byte address of int. memory (also SF registers)
<i>badr</i> :	direct bit address of int. memory (also SF flags)
<i>adr16</i> :	absolute 16-bit address
<i>adr11</i> :	11-bit address relative to 2k page (0 .. 2047 resp. 0 .. 7FFH)
<i>rel</i> :	8-bit address relative to command (-128 .. +127 resp. 80H .. 7FH)
<i>konst8</i> :	8-bit constant (0 .. 255 resp. 0 .. 0FFH)
<i>konst16</i> :	16-bit constant (0.. 65535 resp. 0 .. 0FFFFH)
<i>#B</i> :	Number of bytes used by the command
<i>#Z</i> :	Number of processor cycles for command execution

Addressing modes:

Register addressing: One of *A*, *B*, *Rn* and *DPTR*
Example: *MOV A,R0*

Direct addressing: Any address in internal memory (0..7FH and SFR)
Example: *MOV dadr,80H*
MOV A,SCON

**Indirect addressing
(int. memory):** Address (0..FF) hold in *Ri*
Example: *MOV A,@R0*

**Indirect addressing
(ext. memory):** Address (0..FFFF) hold in *DPTR* or address (0..0FFH) in *Ri*
Example: *MOVX A,@DPTR*

**Indirect, indexed addressing
(program memory):** Address computed from *A* + *DPTR* or *A* + *PC*
Example: *MOVC A,@A+DPTR*

Immediate addressing: Address is the command itself (loading of constants)
Example: *MOV R7,#0FFH* (8-bit value)
MOV DPTR,#8000H (16-bit value)

Transfer commands (copy data):

Copy a byte from or to internal memory / SFR:

MOV	<i>A,Rn</i>	1 B	1 Z	<i>dadr,A</i>	2 B	1 Z
	<i>A,dadr</i>	2 B	1 Z	<i>dadr,Rn</i>	2 B	2 Z
	<i>A,@Ri</i>	1 B	1 Z	<i>dadr,dadr</i>	3 B	2 Z
	<i>A,#konst8</i>	2 B	1 Z	<i>dadr,@Ri</i>	2 B	2 Z
				<i>dadr,#konst8</i>	3 B	2 Z
	<i>Rn,A</i>	1 B	1 Z	<i>@Ri,A</i>	1 B	1 Z
	<i>Rn,dadr</i>	2 B	2 Z	<i>@Ri,dadr</i>	2 B	2 Z
	<i>Rn,#konst8</i>	2 B	1 Z	<i>@Ri,#konst8</i>	2 B	1 Z

Copy a byte from or to external memory:

MOVX	<i>A,@Ri</i>	1 B	2 Z	<i>@Ri,A</i>	1 B	2 Z
	<i>A,@DPTR</i>	1 B	2 Z	<i>@DPTR,A</i>	1 B	2 Z

Copy a byte from program memory to Accumulator:

MOVC	<i>A,@A+PC</i>	1 B	2 Z	
	<i>A,@A+DPTR</i>	1 B	2 Z	

Copy a bit from Carry to internal memory or vice versa:

MOV	<i>C,badr</i>	2 B	1 Z	
	<i>badr,C</i>	2 B	2 Z	

Copy a 16-bit constant to the DPTR register:

MOV	<i>DPTR,#konst16</i>	3 B	2 Z	
------------	----------------------	-----	-----	--

Transfer commands (exchange data):

Exchange Accumulator with byte in internal memory:

<i>XCH</i>	<i>A,Rr</i>	1 B	1 Z	
	<i>A,dadr</i>	2 B	1 Z	
	<i>A,@Ri</i>	1 B	1 Z	

Exchange bits 0..3 (digit) of Accumulator with Register:

<i>XCHD</i>	<i>A,@Ri</i>	1 B	1 Z	
--------------------	--------------	-----	-----	--

Exchange bits 0..3 and 4..7 of Accumulator:

<i>SWAP</i>	<i>A</i>	1 B	1 Z	
--------------------	----------	-----	-----	--

Transfer commands (copy data from or to stack):

Copy a byte from internal memory or SFR to stack:

<i>PUSH</i>	<i>dadr</i>	2 B	2 Z	
--------------------	-------------	-----	-----	--

Copy a byte from stack to internal memory or SFR:

<i>POP</i>	<i>dadr</i>	2 B	2 Z	
-------------------	-------------	-----	-----	--

Other commands:

No operation:

<i>NOP</i>		1 B	1 Z	
-------------------	--	-----	-----	--

Logical commands using bytes:

The result of the logical operation will always be hold in the left operand*:

ANL	<i>A,Rn</i>	1 B	1 Z	<i>dadr,A</i>	2 B	1 Z
ORL	<i>A,dadr</i>	2 B	1 Z	<i>dadr,#konst8</i>	3 B	2 Z
XRL	<i>A,@Ri</i>	1 B	1 Z			
	<i>A,#konst8</i>	2 B	1 Z			

Invert all bits of the Accumulator:

CPL	<i>A</i>	1 B	1 Z	
------------	----------	-----	-----	--

Clear the Accumulator*:

CLR	<i>A</i>	1 B	1 Z	
------------	----------	-----	-----	--

*: If the Accumulator is the destination, the *P*-Bit in PSW will be affected.

Logical commands using bits:

The result of the logical operation will always be hold in the Carry:

ANL	<i>C,badr</i>	2 B	2 Z	
ORL	<i>C,/badr</i>	2 B	2 Z	

Invert a bit:

CPL	<i>C</i>	1 B	1 Z	
	<i>badr</i>	2 B	1 Z	

Clear a bit:

CLR	<i>C</i>	1 B	1 Z	
	<i>badr</i>	2 B	1 Z	

Set a bit:

SETB	<i>C</i>	1 B	1 Z	
	<i>badr</i>	2 B	1 Z	

Arithmetic commands:

The result of the logical operation will always be hold in the Accumulator:

ADD	<i>A,Rn</i>	1 B	1 Z	
ADDC	<i>A,dadr</i>	2 B	1 Z	
SUBB	<i>A,@Ri</i>	1 B	1 Z	
	<i>A,#konst8</i>	2 B	1 Z	

In PSW the bits *C*, *OV*, *AC* and *P* will be affected according the result.

ADD(C): *C* = 1: Result >255

SUBB: *C* = 1: [Dest byte] < [Src byte] (e.g. [A] < konst8)

C = 0: [Dest byte] ≥ [Src byte] (e.g. [A] ≥ konst8)

The addressed byte will be incremented or decremented by 1:

INC	<i>A</i>	1 B	1 Z	
DEC	<i>Rn</i>	1 B	1 Z	
	<i>@Ri</i>	1 B	1 Z	
	<i>dadr</i>	2 B	1 Z	

The DPTR register (16 bit) will be incremented by 1:

INC	<i>DPTR</i>	1 B	1 Z	
------------	-------------	-----	-----	--

The Accumulator *A* will be multiplied by register *B* – result in *B,A*:

MUL	<i>AB</i>	1 B	4 Z	
------------	-----------	-----	-----	--

In PSW the *OV* bit will be set if the result is larger than 255, *C* is set to 0.

The Accumulator *A* will be divided by register *B* - quotient in *A*, remainder in *B*:

DIV	<i>AB</i>	1 B	4 Z	
------------	-----------	-----	-----	--

In PSW the *OV* bit will be set if *B* = 0, *C* is set to 0.

Decimal adjust after an addition of BCD numbers in *A*:

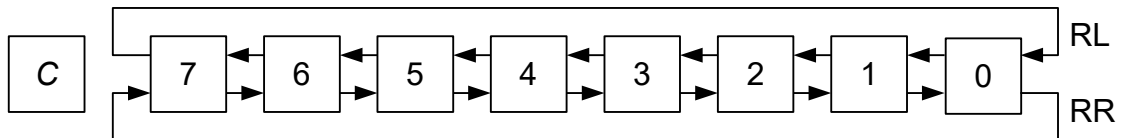
DA	<i>A</i>	1 B	1 Z	
-----------	----------	-----	-----	--

In PSW the *C* bit (Carry) will be affected.

Shift commands

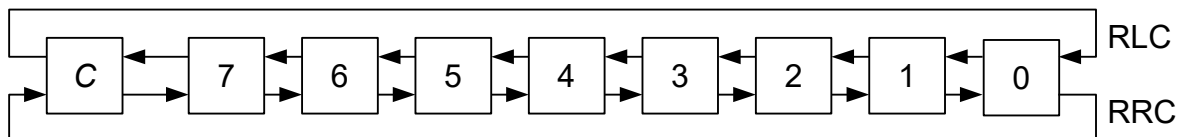
Rotate all bits of the Accumulator by one to the left or right (without C):

<i>RL</i> <i>RR</i>	A	1 B	1 Z	
--------------------------------------	---	-----	-----	--



Rotate all bits of the Accumulator and the Carry by one to the left or right:

<i>RLC</i> <i>RRC</i>	A	1 B	1 Z	
--	---	-----	-----	--



Unconditional jump commands:

Jump to an absolute 16-bit address:

<i>LJMP</i>	<i>adr16</i>	3 B	2 Z	
--------------------	--------------	-----	-----	--

Jump to 11-bit address relative to 2k page boundary:

<i>AJMP</i>	<i>adr11</i>	2 B	2 Z	
--------------------	--------------	-----	-----	--

Jump to an address relative to the command (-128 .. +127):

<i>SJMP</i>	<i>rel</i>	2 B	2 Z	
--------------------	------------	-----	-----	--

Jump to an indirect indexed address:

<i>JMP</i>	<i>@A+DPTR</i>	1 B	2 Z	
-------------------	----------------	-----	-----	--

Unconditional jump to a subroutine:

Jump to an absolute 16-bit address:

<i>LCALL</i>	<i>adr16</i>	3 B	2 Z	
---------------------	--------------	-----	-----	--

Jump to an 11-bit address relative to 2k page boundary:

<i>ACALL</i>	<i>adr11</i>	2 B	2 Z	
---------------------	--------------	-----	-----	--

Return from a subroutine:

Return from a regular routine:

<i>RET</i>		1 B	2 Z	
-------------------	--	-----	-----	--

Return from interrupt routine:

<i>RETI</i>		1 B	2 Z	
--------------------	--	-----	-----	--

Conditional jump commands:

Conditional relative jump depending on Carry bit:

JC JNC	<i>rel</i>	2 B	2 Z	
-------------------------	------------	-----	-----	--

Conditional relative jump depending on contents of Accumulator (=0 or ≠0):

JZ JNZ	<i>rel</i>	2 B	2 Z	
-------------------------	------------	-----	-----	--

Conditional relative jump depending on the status of a bit:

JB JNB JBC	<i>badr,rel</i>	3 B	2 Z	
---------------------------------------	-----------------	-----	-----	--

Conditional relative jump if Accumulator is unequal to data byte or
Accumulator / Register / Data byte is unequal to constant:

CJNE	<i>A,dadr,rel</i>	3 B	2 Z	
	<i>A,#konst8,rel</i>	3 B	2 Z	
	<i>Rn,#konst8,rel</i>	3 B	2 Z	
	<i>@Ri,#konst8,rel</i>	3 B	2 Z	

C = 1: [Dest byte] < [Src byte] (e.g. [A] < konst8)

C = 0: [Dest byte] ≥ [Src byte] (e.g. [A] ≥ konst8)

The Register / Data byte will be decremented by 1, conditional relative jump if ≠0:

DJNZ	<i>Rn,rel</i>	2 B	2 Z	
	<i>dadr,rel</i>	3 B	2 Z	